

PHARMACY

MANAGEMENT SYSTEM

OOP Lab Project · Team Briefing Document

COURSE	TEAM SIZE	DOCUMENT TYPE	SEMESTER
Object Oriented Programming CSP-122	5 Members 4 Developers + 1 Docs	Team Briefing & Workload Distribution	Spring 2026 IoBM

What This Document Covers

- System overview — who uses the system and what each user can do
- Complete login and menu workflow — how the system starts and navigates
- Three-tier user architecture — Admin, Employee, Customer
- Alternative medicine feature — how out-of-stock suggestions work
- Full workload distribution — which team member builds which class
- GUI implementation plan — when and how to add it, who does it
- Build timeline — phase by phase from week 1 to submission
- How all classes connect — the complete data flow

01 — System Overview

What is this system and who uses it?

The Big Picture

The Pharmacy Management System is a **desktop console application** built in Java. It is **not a website** — it runs as a program on a computer. Think of it like the software a real pharmacy uses behind the counter. It manages medicines, handles sales, generates bills, tracks stock, and alerts staff when medicines expire or run low.

USER TYPE	LOGIN REQUIRED?	ACCESS LEVEL	REAL WORLD ANALOGY
-----------	-----------------	--------------	--------------------

Admin	Yes — Username + Password	Full access. Add, delete, update medicines, change prices, view reports, manage staff accounts
Employee	Yes — Username + Password	Restricted. Can sell, search, view inventory only. Can delete or change prices
Customer	No — Direct access	Browse only. Search medicines, check prices, see availability, get alternatives if out of stock

Key Point: Customer gets direct access because they are just browsing — like walking into a real pharmacy and looking at the shelves. Admin and Employee need credentials because they touch backend data — prices, stock, records.

02 — Login & Menu Workflow

How the system starts and how users navigate it

Startup Flow

Program Launches



Alerts run automatically — expired medicines + low stock shown before menu



Welcome Screen: "Who are you? [1] Admin [2] Employee [3] Customer"



Admin selected → Enter Username + Password → Max 3 attempts → Admin Menu

Employee selected → Enter Username + Password → Max 3 attempts → Employee Menu

Customer selected → No login needed → Customer Menu directly

ADMIN MENU	EMPLOYEE MENU	CUSTOMER MENU
• Add new medicine • Update medicine details • Delete medicine • Change prices • View full inventory • View all sales reports • Manage employee accounts • View activity log • Configure alert thresholds	• Search medicine by name • Search medicine by ID • Sell medicine • Generate and print bill • View inventory (read only) • Check stock levels • View own sales history	• Search medicine by name • View available medicines • Check price • Check availability • See alternatives if out of stock • Place order request • View medicine category/formula

03 — Alternative Medicine Feature

Your team's original idea — how it works technically

When a customer searches for a medicine that is **out of stock or not found**, instead of just showing 'not available', the system searches the inventory for medicines with the **same formula category** and suggests them as alternatives.

How It Works	Technical Implementation
Each medicine in the inventory has a formulaCategory field — for example: 'Paracetamol', 'Amoxicillin', 'Omeprazole'. When a search returns out-of-stock or no result, the system loops through the entire inventory and finds medicines sharing the same formulaCategory.	One extra field in Medicine class: String formulaCategory One extra method in Medicine class: findAlternatives(formulaCategory) → loops array → returns matches with same category → displays as suggestions

Customer searches:	"Panadol"
Result:	Out of stock
System checks:	formulaCategory = "Paracetamol"
Inventory scan:	Found "Calpol" [Paracetamol] – qty: 50 Found "Paracet" [Paracetamol] – qty: 30
System displays:	"Panadol unavailable. Alternatives: Calpol, Paracet"

04 — Team Workload Distribution

Who builds what — equal responsibility, full ownership

Ground Rule: Each member fully owns their classes. They build them, they test them, and they can explain every line in the team meeting and to the teacher. No one submits work they cannot explain.

TEAM LEADER	Member 1 — You
Medicine.java	FileManager.java
RESPONSIBILITIES ■ All medicine fields — name, company, price, expiry, quantity, formulaCategory, prescriptionRequired ■ CRUD operations — add, update, delete, search by name and ID ■ Alternative medicine suggestion — findAlternatives(formulaCategory) ■ All file read/write — inventory.txt, sales_log.txt, activity_log.txt, backups ■ Project folder and file structure setup for the whole team ■ Guide and unblock other members when they get stuck <i>You understand the full picture. These two classes are what everything else depends on. If these are solid, everyone else works smoothly.</i>	

SALES & BILLING

Member 2

Sales.java

Bill.java

SalesReport.java

RESPONSIBILITIES

- sellMedicine(id, quantity) — main sell method, checks stock before proceeding
- applyDiscount(percent) — admin only, reduces bill total
- processRefund(billID) — reverses a sale, restores stock
- Bill generation — billID, medicine name, quantity, price, discount, total, date
- printBill() — formatted receipt display
- Daily, weekly, monthly sales summaries
- Best selling medicine tracker and profit/loss calculation

Sales logic is self-contained. Once Medicine and FileManager are ready this member works independently with no dependencies on other members.

ACCESS CONTROL

Member 3

User.java (abstract)

Admin.java

Staff.java

Customer.java

RESPONSIBILITIES

- Abstract User class — username, password, role, login attempts
- abstract login() method — subclasses must implement it
- Admin extends User — full access, changePrice(), deleteMedia(), viewActivityLog()
- Staff extends User — restricted access, sell and view only
- Customer class — search, view, order, receive alternative suggestions
- Login logout — max 3 wrong attempts before session locks
- Activity logging — every admin action recorded with timestamp

Access control is its own independent layer. Showcases Abstraction, Inheritance, and Polymorphism in one clean package — strong OOP demonstration.

AlertSystem.java

DateHelper.java

Menu.java

RESPONSIBILITIES

- AlertSystem — check expiry and low stock automatically on every startup
- Expiry warning threshold — alert X days before expiry date, not just on the day
- DateHelper — utility class for date comparisons and formatting
- Menu.java — the main controller, all Scanner input and screen output
- Startup sequence — load inventory, run alerts, show welcome screen
- Shutdown sequence — save inventory, generate dated backup file
- Routes every user choice to the correct class and method

This is the glue of the project. This member must understand everyone's work to connect it all — assign someone reliable and communicative.

DOCUMENTATION

Project Proposal

Final Report

UML Diagram

Presentation

RESPONSIBILITIES

- Project proposal document — problem statement, objectives, scope
- UML class diagram — hierarchy showing all classes and relationships
- Final report — introduction, class descriptions, screenshots, conclusions
- Presentation slides — one slide per major feature, demo walkthrough
- Read the blueprint PDF and write each class description in her own words
- Coordinate with team for screenshots once console version is working

Every real software team has a documentation role. She learns the full system by writing about it without being burdened with code. She becomes the team's communicator.

05 — GUI Implementation Plan

When to add it, who does it, and how it connects to existing backend

GUI means **Graphical User Interface** — actual windows, buttons, and forms instead of a black terminal screen. In Java this is done using **Java Swing** (most likely what your teacher will introduce). This is a **desktop application** — not a website, not a browser. Think of it like a Windows program.

The Most Important Point About GUI: Your backend code does NOT change at all when you add GUI. The Medicine, Sales, FileManager, AlertSystem classes stay exactly the same. GUI simply replaces the Menu class — instead of Scanner input, you click buttons. The buttons call the exact same backend methods. This is why Menu was designed separately from all logic classes.

CONSOLE VERSION (Now)		GUI VERSION (Later)
User types numbers in terminal ↓ Menu.java (Scanner) ↓ Medicine / Sales / FileManager (backend unchanged)	→ Replace Menu only	User clicks buttons in a window ↓ GUI Windows (Swing) ↓ Medicine / Sales / FileManager (same backend, unchanged)

GUI Work Assignment

Member 4	Builds all GUI windows — Main login window with three role buttons, Admin panel with forms and tables, Employee panel with search and sell, Customer panel with search and alternatives display.
Member 2	Assists Member 4 with the Sales and Bill display panels since they built the backend for it — they know exactly what data to show.
Rule	Build GUI only AFTER the console version works completely and has been tested. GUI is a skin over working logic — never build both at the same time.

06 — Build Timeline

Phase by phase from week 1 to final submission

PHASE 1	Backend Foundations	Week 1
- Member 1 (You) → Medicine.java fully complete with all fields and methods - Member 1 (You) → FileManager.java fully complete with all read/write operations - Member 3 → User (abstract), Admin, Staff, Customer classes - Member 5 → Begin project proposal document - Goal: Core data layer done, access control done		

PHASE 2 Sales, Alerts & Integration

Week 2

- Member 2 → Sales.java, Bill.java complete (needs Phase 1 done first)
- Member 4 → AlertSystem.java, DateHelper.java complete
- Member 2 → SalesReport.java (daily/weekly/monthly summaries)
- Member 5 → Finalize proposal, begin UML diagram
- Goal: All backend classes complete and unit tested individually

PHASE 3 Console Integration & Testing

Week 3

- Member 4 → Build Menu.java — connect all classes with Scanner interface
- Full Team → Integration testing — run all scenarios together
- Member 1 → Fix any connection issues between classes
- Member 5 → Complete UML diagram, begin final report
- Goal: Full console version working end to end — all three user roles tested

PHASE 4 GUI & Final Report

Week 4

- Member 4 → Build GUI windows using Java Swing
- Member 2 → Assist with Sales and Bill display panels
- Full Team → Test complete GUI version thoroughly
- Member 5 → Add screenshots to report, finalize slides
- Goal: GUI version complete, report done, everyone can explain their part

PHASE 5 Submission Preparation

Final Week

- Member 1 → Final code review of all classes
- Everyone → Each member practices explaining their own classes
- Member 5 → Submit proposal and final report
- Full Team → Prepare project demo walkthrough
- Goal: Confident submission — every member can answer teacher questions

07 — How All Classes Connect

The complete data flow for every major operation

Customer Searches Medicine

- 1 Menu receives search input from Customer
- 2 Menu calls `Medicine.searchByName()`
- 3 If found and in stock → display medicine details and price
- 4 If out of stock → `Medicine.findAlternatives(formulaCategory)`
- 5 Alternatives displayed: same formula family, different brand

Employee Sells Medicine

- 1 Menu receives sell request from Employee
- 2 `Sales.sellMedicine(id, quantity)` called
- 3 Sales checks Medicine stock — if insufficient, show error
- 4 Sales creates new Bill object with all transaction details
- 5 Sales calls Medicine to reduce stock by quantity sold
- 6 Sales calls `FileManager.logSale(bill)` to save to `sales_log.txt`
- 7 `Bill.printBill()` displays formatted receipt on screen

Admin Changes Price

- 1 Menu checks `currentUser instanceof Admin`
- 2 If not Admin → access denied message shown
- 3 `Admin.changePrice(medicineID, newPrice)` called
- 4 Medicine record updated in the inventory array
- 5 `FileManager.saveInventory()` writes updated array to file
- 6 `FileManager.logActivity('Price changed for X by Admin')` records action

Program Starts Up

- 1 FileManager.loadInventory() reads inventory.txt into Medicine array
- 2 AlertSystem.checkExpiry() scans all medicines for upcoming expiry dates
- 3 AlertSystem.checkLowStock() scans all medicines below minimum threshold
- 4 All alerts printed to screen before the welcome menu appears
- 5 Welcome screen shown: Select Admin, Employee, or Customer

Program Shuts Down

- 1 FileManager.saveInventory() writes current Medicine array to inventory.txt
- 2 FileManager.generateBackup() creates inventory_DD-MM-YYYY.txt in backup folder
- 3 All unsaved activity log entries flushed to activity_log.txt
- 4 Program exits cleanly

08 — Team Rules & Final Notes

Things every member must know before the first line of code

- 1 One **public class** per file — filename must exactly match the public class name.
- 2 All other classes go in the **same file** as their public class — no ClassNotFoundException.
- 3 All fields **private** — always use getters and setters. No exceptions.
- 4 FileManager is the **only class** that reads or writes files — no other class touches files directly.
- 5 Every member must be able to **explain their own classes** to the teacher without help.
- 6 Build the **console version first** — GUI comes only after the console works end to end.
- 7 Use **try-catch** around every file operation — files may not exist on the first run.
- 8 AlertSystem always runs **before the menu appears** — check stock and expiry on startup.
- 9 Every sale creates a Bill object and gets logged **immediately** — never batch or delay.
- 10 The team leader reviews all code **before integration** — catch problems early.

Pharmacy Management System · OOP Lab Project · Team Briefing Document

Made by Syed Muhammad Ahmed — Executive Director, TPX Orlando, Florida

